

Kotlin & Jetpack Compose Concepts Guide - Vol. 3

ViewModels & AppModule | ViewModel Disposal
Navigation Patterns | iOS / Cross-Platform

Continues from Vol. 2.

Covers architecture decisions, navigation patterns,
and cross-platform considerations.

Table of Contents

1. Should ViewModels go in AppModule?
2. What DOES go in AppModule vs what does not
3. How @HiltViewModel registers itself (no AppModule needed)
4. ViewModel Disposal - When is it destroyed?
5. The back stack and ViewModel lifetime step by step
6. Navigation Patterns - Three Approaches
7. Approach 1: Callbacks (recommended for most cases)
8. Approach 2: NavController passed directly (anti-pattern)
9. Approach 3: ViewModel navigation events via Channel
10. Why Channel and not StateFlow for navigation events
11. Which navigation approach to use when
12. Can this app run on iOS?
13. What blocks iOS: Hilt is Android-only
14. Options for cross-platform: KMP, Flutter, React Native
15. Can this project be configured for iOS? (KMP migration)
16. Full summary and learning path recommendation

1. Should ViewModels go in AppModule?

Short answer: No. Never put ViewModels in AppModule. This is one of the most common points of confusion when learning Hilt, so here is a clear explanation of why.

The rule

```
// AppModule.kt -- CORRECT
// Only repositories, API clients, databases, and other services go here
@Module
@InstallIn(SingletonComponent::class)
abstract class AppModule {
    @Binds @Singleton
    abstract fun bindProductRepository(impl: ProductRepositoryImpl): IProductRepository

    @Binds @Singleton
    abstract fun bindCartRepository(impl: CartRepositoryImpl): ICartRepository
}

// ProductsViewModel.kt -- CORRECT
// @HiltViewModel is all that is needed. No entry in AppModule required.
@HiltViewModel
class ProductsViewModel @Inject constructor(
    @Named("real") private val productRepository: IProductRepository,
) : ViewModel()
```

[!] If you try to register a ViewModel in AppModule with @Binds or @Provides, Hilt will throw a compile-time error. ViewModels have their own dedicated Hilt component (ViewModelComponent) and must not be installed into SingletonComponent.

Why ViewModels are different from repositories

Repositories are services -- they are created once and shared. ViewModels are screen-specific -- each screen instance gets its own ViewModel instance, and the ViewModel must be tied to the screen's lifecycle (NavBackStackEntry), not the app's lifetime. Hilt handles this automatically through ViewModelComponent.

```
// Hilt component hierarchy (simplified):
//
// SingletonComponent      <- @InstallIn here = app lifetime (repositories)
//   |
//   ActivityRetainedComponent <- survives rotation, tied to Activity
//     |
//     ViewModelComponent    <- @HiltViewModel goes here automatically
//       |
//       ActivityComponent   <- tied to Activity lifecycle
//         |
//         FragmentComponent
//
// AppModule uses SingletonComponent.
// ViewModels live in ViewModelComponent -- a different level entirely.
```

2. What DOES go in AppModule vs what does not

GOES in AppModule	Does NOT go in AppModule
Repository interface bindings (@Binds IProductRepository)	ViewModels (@HiltViewModel handles these)
Third-party objects (@Provides Retrofit, OkHttp, Gson)	Composable functions (these are not injected)
Database instance (@Provides Room AppDatabase)	Activities / Fragments (@AndroidEntryPoint handles these)
DAO interfaces (@Provides CartDao from AppDatabase)	NavController (created inside the composable)
DataStore instance (@Provides DataStore<Preferences>)	UI state data classes (these are plain Kotlin classes)
Shared helpers / managers (analytics, logging, etc.)	Models / data classes (no DI needed, just instantiate them)

The guiding question

Ask yourself: does this object need to be shared across multiple ViewModels or screens? If yes, it probably belongs in AppModule as a @Singleton. If it is specific to one screen, it either goes in the ViewModel constructor directly (injected by Hilt) or is created locally inside the composable.

```
// Example: adding a real API service to AppModule

@Module
@InstallIn(SingletonComponent::class)
object NetworkModule {

    // @Provides (not @Binds) because Retrofit is a third-party class
    // we cannot annotate with @Inject
    @Provides
    @Singleton
    fun provideRetrofit(): Retrofit {
        return Retrofit.Builder()
            .baseUrl("https://api.yourstore.com/")
            .addConverterFactory(GsonConverterFactory.create())
            .build()
    }

    @Provides
    @Singleton
    fun provideApiService(retrofit: Retrofit): ApiService {
        // Hilt sees Retrofit is needed -> looks in the module -> finds provideRetrofit()
        // Hilt calls provideRetrofit() first, then passes result here automatically
        return retrofit.create(ApiService::class.java)
    }
}

// Then ProductRepositoryImpl can receive ApiService via @Inject:
```

```
class ProductRepositoryImpl @Inject constructor(  
    private val apiService: ApiService, // Hilt provides this from NetworkModule  
) : IProductRepository
```

3. How @HiltViewModel registers itself (no AppModule needed)

When you annotate a ViewModel with `@HiltViewModel`, KSP generates a factory class for it automatically at compile time. You never write this factory -- it is generated for you. Here is what happens behind the scenes.

Compile-time chain

Step 1: You write this

```
@HiltViewModel
class CartViewModel @Inject constructor(
    private val cartRepository: ICartRepository,
) : ViewModel()
```

Step 2: KSP generates a factory (in build/generated/, you never see or edit this)

```
class CartViewModel_HiltModules_KeyModule_ProvideFactory {
    fun provide(): CartViewModel {
        val cartRepo = cartRepositoryProvider.get() // resolved from AppModule @Binds
        return CartViewModel(cartRepository = cartRepo)
    }
}
```

Step 3: `hiltViewModel()` looks up the generated factory at runtime

```
@Composable
fun CartScreen(viewModel: CartViewModel = hiltViewModel()) {
    // hiltViewModel() asks Hilt: "give me a CartViewModel"
    // Hilt finds the generated factory
    // Factory asks for ICartRepository -> Hilt resolves to CartRepositoryImpl (from AppModule)
    // CartViewModel is created with CartRepositoryImpl injected
}
```

[NOTE] Flutter comparison: in Flutter you manually call `ExampleController(repository: repository)` inside a widget. In Hilt, `hiltViewModel()` does that construction call for you -- automatically, with all dependencies resolved, in one line.

The full picture

```
AppModule (@Binds)
  "when ICartRepository is needed, provide CartRepositoryImpl"
  |
  v
CartViewModel_HiltModules_KeyModule (generated by KSP)
  "to build CartViewModel, I need ICartRepository"
  "ICartRepository -> AppModule -> CartRepositoryImpl"
  |
  v
hiltViewModel() call in CartScreen composable
  "give me CartViewModel" -> factory creates it -> returns fully injected instance
  |
  v
CartScreen gets CartViewModel with CartRepositoryImpl already inside it
```

4. ViewModel Disposal - When is it destroyed?

This is one of the most important things to understand about ViewModel lifecycle. The ViewModel is NOT destroyed every time a screen goes off-screen. It depends on whether the screen is still in the back stack.

The three situations

Situation	Back stack entry?	ViewModel destroyed?
User navigates TO another screen (new screen pushed on top)	Still alive (current screen is below)	NO -- ViewModel stays in memory
User presses Back (current screen popped)	Removed from back stack	YES -- <code>onCleared()</code> called
Device is rotated (configuration change)	Survives rotation (<code>NavBackStackEntry</code> preserved)	NO -- same instance reused
App goes to background (user switches apps briefly)	Still alive	NO -- ViewModel stays
Android kills the process (app in background too long)	Destroyed by OS	YES -- everything in memory is gone
User force-closes the app	Destroyed	YES -- process is killed

[NOTE] ViewModel lifetime = NavBackStackEntry lifetime. As long as the screen exists anywhere in the back stack (even hidden behind other screens), the ViewModel stays alive. Only when the entry is removed does the ViewModel get cleared.

5. The Back Stack and ViewModel Lifetime Step by Step

Let us trace through a realistic user journey in the Online Store app and see exactly which ViewModels are alive at each step.

User journey trace

```
App starts
  Back stack: [Products]
  ViewModels alive: ProductsViewModel (created, init -> load() called)

User taps a product card
  Back stack: [Products, ProductDetail]
  ViewModels alive: ProductsViewModel (still alive!), ProductDetailViewModel (just created)

User taps cart icon from ProductDetail
  Back stack: [Products, ProductDetail, Cart]
  ViewModels alive: ProductsViewModel, ProductDetailViewModel, CartViewModel (just created)
  CartViewModel.load() called via LaunchedEffect

User presses Back from Cart
  Back stack: [Products, ProductDetail]
  ViewModels alive: ProductsViewModel, ProductDetailViewModel
  CartViewModel.onCleared() called -> CartViewModel DESTROYED

User taps "Add to Cart" button
  Back stack: [Products, ProductDetail]
  ProductDetailViewModel.addToCart() -> CartRepositoryImpl._cartItems updated (Singleton)

User presses Back from ProductDetail
  Back stack: [Products]
  ProductDetailViewModel.onCleared() called -> ProductDetailViewModel DESTROYED

User taps cart icon from Products
  Back stack: [Products, Cart]
  CartViewModel CREATED AGAIN (brand new instance)
  LaunchedEffect -> load() -> cartRepository.getCartItems() -> shows the item added earlier
  (CartRepositoryImpl is @Singleton so the data is still there)
```

Why this design is intentional

You might wonder: why is the ViewModel destroyed when you press Back? Why not keep it alive to avoid reloading? The answer is memory. If every screen you visited kept its ViewModel forever, the app would consume more and more memory over time. Tying ViewModel lifetime to back stack lifetime is the right tradeoff: data is reloaded when needed (cheap), and memory is freed when the screen is gone (important).

onCleared() -- the ViewModel's cleanup hook

```
@HiltViewModel
```



```
class CartViewModel @Inject constructor(  
    private val cartRepository: ICartRepository,  
    ) : ViewModel() {  
  
    // Called automatically by Hilt/ViewModelStore when the NavBackStackEntry is removed.  
    // Use this to cancel subscriptions, close database cursors, stop timers, etc.  
    // viewModelScope coroutines are cancelled automatically -- no need to cancel them here.  
    override fun onCleared() {  
        super.onCleared()  
        // example: EventBus.unregister(this)  
    }  
}
```

[NOTE] Flutter equivalent: dispose() in a StatefulWidget's State class. In Flutter you manually call dispose() on controllers. In Android, viewModelScope coroutines are cancelled automatically -- you only need onCleared() for non-coroutine cleanup.

6. Navigation Patterns - Three Approaches

There are three ways to handle navigation in a Jetpack Compose app. Each has its place. Understanding when to use each one is important.

Approach	How it works	When to use
1. Callbacks (recommended)	Screen exposes onXxx lambda params. AppNavigation wires them to navController.	User taps a button and navigates immediately. Most common case.
2. NavController directly (avoid)	NavController is passed as a parameter into the screen composables.	Never. This is an anti-pattern. Avoid completely.
3. ViewModel Channel events	ViewModel emits navigation events via Channel. Screen observes and calls callback.	Navigation triggered after an async operation (API call, DB write, etc.).

7. Approach 1: Callbacks (Recommended for Most Cases)

The screen knows nothing about navigation. It exposes lambda parameters for every action that might trigger navigation. AppNavigation wires these lambdas to the actual navController calls. This is what the Online Store app uses.

How it works in the app

```
// ProductsScreen.kt -- no NavController, no knowledge of routes
@Composable
fun ProductsScreen(
    onProductClick: (Int) -> Unit, // expose the action, not the destination
    onCartClick: () -> Unit,
    viewModel: ProductsViewModel = hiltViewModel(),
) {
    // ...
    ProductCard(
        product = product,
        onClick = { onProductClick(product.id) } // fires the callback, nothing else
    )
    IconButton(onClick = onCartClick) { ... }
}

// AppNavigation.kt -- the ONLY place that knows about routes
composable(AppRoute.Products.route) {
    ProductsScreen(
        onProductClick = { productId ->
            navController.navigate(AppRoute.ProductDetail.createRoute(productId))
        },
        onCartClick = {
            navController.navigate(AppRoute.Cart.route)
        },
    )
}
```

[OK] This is Google's officially recommended pattern for Compose navigation. Screens are reusable and fully decoupled from the navigation infrastructure.

Why this is good

```
// ProductsScreen does not import NavController, does not know AppRoute exists.
// You could reuse ProductsScreen in a completely different app and just wire
// different callbacks -- the screen itself never changes.

// Flutter comparison: equivalent to a widget that calls onTap() callbacks
// instead of calling Navigator.of(context).pushNamed() directly.
// The widget tree decides where tapping leads, not the widget itself.
```

8. Approach 2: NavController Directly (Anti-Pattern)

Passing NavController directly into a screen composable as a parameter feels convenient but creates serious problems.

What it looks like

```
// DO NOT DO THIS
@Composable
fun ProductsScreen(navController: NavController) {
    ProductCard(
        onClick = { navController.navigate("product_detail/${product.id}") }
    )
}
```

[!] This is an anti-pattern. Avoid it completely.

Why it is wrong

Problem 1: Tight coupling

ProductsScreen now depends on NavController.
 You cannot use ProductsScreen without a NavController.
 Testing requires a real or fake NavController.

Problem 2: Route strings leak into screens

The screen now contains "product_detail/\${product.id}" -- a hardcoded route string.
 If you rename the route, you must find and update every screen that uses it.
 With callbacks, only AppNavigation needs to change.

Problem 3: No single source of truth for navigation

Navigation logic is scattered across every screen file.
 With callbacks, ALL navigation logic is in ONE place: AppNavigation.kt.

```
// Flutter equivalent of this anti-pattern:
// Passing BuildContext deep into widgets just to call Navigator.of(context).push()
// Flutter best practice is the same: bubble navigation up via callbacks.
```

9. Approach 3: ViewModel Navigation Events via Channel

Approaches 1 and 2 handle user-triggered navigation (button tap -> navigate). But sometimes navigation must happen AFTER an async operation completes -- for example, only navigate to the order confirmation screen after the API call succeeds. For this, the ViewModel emits a navigation event through a Channel.

Implementation

```
// Step 1: Define navigation events as a sealed class
sealed class OrderNavigationEvent {
    object NavigateToConfirmation : OrderNavigationEvent()
    object NavigateBack : OrderNavigationEvent()
}

// Step 2: Add a Channel to the ViewModel
@HiltViewModel
class OrderViewModel @Inject constructor(
    private val orderRepository: IOrderRepository,
    private val cartRepository: ICartRepository,
) : ViewModel() {

    private val _state = MutableStateFlow<OrderState>(OrderState.Initial)
    val state: StateFlow<OrderState> = _state.asStateFlow()

    // Channel = one-shot event delivery (each event consumed exactly once)
    private val _navigationEvent = Channel<OrderNavigationEvent>()
    val navigationEvent = _navigationEvent.receiveAsFlow()

    fun placeOrder() {
        viewModelScope.launch {
            _state.value = OrderState.Loading
            try {
                val order = orderRepository.placeOrder(cartRepository.getCartItems())
                cartRepository.clearCart()
                _state.value = OrderState.Completed(order)
                // Navigate ONLY after the business logic succeeds
                _navigationEvent.send(OrderNavigationEvent.NavigateToConfirmation)
            } catch (e: Exception) {
                _state.value = OrderState.Error(e.message ?: "Unknown error")
            }
        }
    }
}

// Step 3: Screen observes events and calls the provided callback
@Composable
fun OrderScreen(
    onNavigateToConfirmation: () -> Unit,
    viewModel: OrderViewModel = hiltViewModel(),
) {
```

```

    LaunchedEffect(Unit) {
        viewModel.placeOrder()
    }

    // Collect navigation events -- each event fires the callback once
    LaunchedEffect(Unit) {
        viewModel.navigationEvent.collect { event ->
            when (event) {
                is OrderNavigationEvent.NavigateToConfirmation -> onNavigateToConfirmation()
                is OrderNavigationEvent.NavigateBack -> { /* handle */ }
            }
        }
    }

    // Render loading/error/completed state as normal...
}

// AppNavigation.kt -- same callback pattern, nothing changes here
composable(AppRoute.Order.route) {
    OrderScreen(
        onNavigateToConfirmation = {
            navController.navigate(AppRoute.Confirmation.route) {
                popUpTo(AppRoute.Products.route)
            }
        },
    )
}

```

10. Why Channel and Not StateFlow for Navigation Events

This is a subtle but important point. Using StateFlow for navigation events causes a bug that is hard to debug. Here is exactly why Channel is the right choice.

The StateFlow replay problem

```
// WRONG -- using StateFlow for navigation
private val _navEvent = MutableStateFlow<OrderNavigationEvent?>(null)
val navEvent: StateFlow<OrderNavigationEvent?> = _navEvent.asStateFlow()

// What happens:
// 1. Order placed -> _navEvent.value = NavigateToConfirmation
// 2. Screen collects -> navigates to confirmation (correct)
// 3. User rotates device -> OrderScreen recomposes
// 4. collectAsState() or collect{} starts again
// 5. StateFlow REPLAYS the last value: NavigateToConfirmation
// 6. App navigates to confirmation AGAIN (bug -- user is sent there twice)
```

How Channel solves this

```
// CORRECT -- using Channel for navigation
private val _navEvent = Channel<OrderNavigationEvent>()
val navEvent = _navEvent.receiveAsFlow()

// What happens:
// 1. Order placed -> _navEvent.send(NavigateToConfirmation)
// 2. Screen collects -> consumes the event -> navigates (correct)
// 3. The event is now GONE from the Channel -- it was consumed
// 4. User rotates device -> OrderScreen recomposes
// 5. collect{} starts again but Channel has no pending events
// 6. Nothing happens (correct)
```

StateFlow vs Channel -- the core difference

StateFlow:

- Always has a current value
- New collectors receive the LAST emitted value immediately (replay = 1)
- Perfect for: UI state (loading/error/completed) -- the screen always needs to know the current state when it first appears
- Wrong for: one-shot events like navigation, snackbars, dialogs

Channel:

- Queue of events, each consumed exactly once
- New collectors do NOT receive past events
- Perfect for: navigation events, one-time error messages, snackbars
- Wrong for: persistent state the UI needs to render

[NOTE] Flutter comparison: `StateFlow` is like a `ChangeNotifier` -- notifies all current listeners of the current value. `Channel` is like a `Stream` with no replay -- events are delivered once and gone. In Flutter you would use a `StreamController` with `broadcast:false` for the same one-shot event pattern.

11. Which Navigation Approach to Use When

Q: What triggers the navigation?

User taps a button / link

|

v

Use Approach 1 (Callbacks)

The callback fires immediately on tap.

No async logic involved.

Example: `onProductClick`, `onCartClick`, `onBackClick`

Navigation depends on the result of an async operation

(API call, database write, validation, authentication check)

|

v

Use Approach 3 (ViewModel Channel events)

The ViewModel runs the async logic first.

Only on success (or specific failure) does it send a navigation event.

Example: navigate to confirmation only AFTER order API call succeeds

You are thinking about passing NavController into the screen composable

|

v

Use Approach 1 instead. Always.

Applied to the Online Store app

ProductsScreen -> ProductDetailScreen	Approach 1 (user taps product card)
ProductsScreen -> CartScreen	Approach 1 (user taps cart icon)
ProductDetailScreen -> CartScreen	Approach 1 (user taps cart icon)
CartScreen -> OrderScreen	Approach 1 (user taps Checkout)
OrderScreen -> Products (after order)	Could use Approach 3 (navigate after API) Currently uses Approach 1 watching state Both are valid for this simple case

12. Can This App Run on iOS?

No. Kotlin and Jetpack Compose are Android-only technologies. The code in this project cannot run on iOS without significant changes. This is a fundamental difference from Flutter, which targets both platforms from the same codebase by default.

Why -- the technology stack is Android-specific

Component in this app	Android-only?	Reason
Jetpack Compose	YES	Google's Android UI toolkit. No iOS runtime.
Hilt	YES	Built on Android Application class and Android annotation processor.
ViewModel (androidx)	Partially	Now has a KMP version but needs project restructure.
NavController (Jetpack Nav)	YES	Android-only. KMP alternative: Voyager or Decompose.
@AndroidEntryPoint	YES	Hooks into Android Activity lifecycle directly.
@HiltAndroidApp	YES	Requires Android Application class.
Kotlin language	NO	Kotlin runs on Android, iOS (via KMP), JVM, web.
Coroutines / Flow	NO	Fully KMP-compatible. No changes needed.
data class / sealed class	NO	Pure Kotlin. Works everywhere Kotlin runs.

[!] Hilt is the hardest blocker. It is deeply tied to the Android platform and has no KMP/iOS version. Moving to iOS requires replacing Hilt with a different DI solution (Koin).

13. What Blocks iOS: Hilt is Android-Only

Even if you replaced every other Android-specific component, Hilt alone would prevent the code from running on iOS. Here is why.

What Hilt depends on

```
// Hilt's @HiltAndroidApp requires:
class App : Application() // android.app.Application -- Android SDK class

// Hilt's @AndroidEntryPoint requires:
class MainActivity : ComponentActivity() // androidx.activity.ComponentActivity -- Android SDK

// Hilt's annotation processor (KSP plugin) generates Android-specific Java/Kotlin code
// that imports android.* packages. None of these exist on iOS.
```

The KMP alternative: Koin

Koin is a lightweight DI library that is fully Kotlin Multiplatform compatible. It runs on Android, iOS, desktop, and web. The syntax is different from Hilt but the concept is the same: declare what provides what, inject at use site.

<pre>// Hilt (Android-only) @Module @InstallIn(SingletonComponent::class) abstract class AppModule { @Binds @Singleton abstract fun bindRepo(impl: ProductRepositoryImpl): IProductRepository } @HiltViewModel class ProductsViewModel @Inject constructor(val repo: IProductRepository) : ViewModel() // In composable: hiltViewModel()</pre>	<pre>// Koin (KMP - works on iOS too) val appModule = module { single<IProductRepository> { ProductRepositoryImpl() } single<ICartRepository> { CartRepositoryImpl() } } class ProductsViewModel(val repo: IProductRepository // no annotation needed) : ViewModel() // In composable: koinViewModel<ProductsViewModel>()</pre>
---	---

[NOTE] If you start a new project today with iOS in mind, use Koin instead of Hilt from the start. Migrating an existing Hilt project to Koin is doable but requires touching every @HiltViewModel, @Inject constructor, @Module, and @Binds annotation in the project.

14. Options for Cross-Platform: KMP, Flutter, React Native

Option 1: Kotlin Multiplatform (KMP) + Compose Multiplatform

Write shared Kotlin code for business logic, repositories, and ViewModels. Use Compose Multiplatform for the UI layer on both Android and iOS.

Project structure:

```
shared/
  commonMain/      <- Kotlin code shared by all platforms
    features/
      products/data/    <- repositories (Koin DI)
      products/models/  <- data classes
      products/controller/ <- ViewModels (KMP ViewModel)
  androidMain/      <- Android-specific code (if any)
  iosMain/           <- iOS-specific code (if any)
androidApp/          <- Android entry point (MainActivity)
iosApp/              <- Xcode project (iOS entry point)
```

What changes vs this project:

- Replace Hilt with Koin
- Add the KMP Gradle multiplatform plugin
- Set up an Xcode project for iOS
- Some libraries need KMP versions (check compatibility)
- Compose Multiplatform UI looks identical to Jetpack Compose

What stays the same:

- Kotlin language
- Coroutines and Flow
- data class, sealed class
- The overall clean architecture structure
- Compose UI syntax

[OK] Best option if you want to stay in Kotlin and eventually support iOS. Start a new KMP project from scratch -- do not try to convert this one while learning.

Option 2: Flutter (what you already know)

Runs on: Android, iOS, Web, Desktop -- all from one codebase

Language: Dart

UI: Flutter widgets (equivalent of Compose)

DI: Manual (DependenciesScope pattern you already know)

State: StateController + Freezed (your AI_INSTRUCTIONS.md pattern)

Honest assessment:

You are already proficient in Flutter.

The Online Store features (products, cart, orders, navigation) would take a fraction of the time to build in Flutter compared to learning Kotlin first.

Flutter is a completely valid production choice for cross-platform apps.

Option 3: React Native

Runs on: Android, iOS

Language: JavaScript / TypeScript

Not relevant given your Kotlin/Dart background -- skip this option.

Decision table

	Kotlin/Jetpack (this app)	KMP + Compose Multi	Flutter
iOS support	No	Yes (with Koin)	Yes (built-in)
Language	Kotlin	Kotlin	Dart
UI framework	Jetpack Compose	Compose Multiplatform	Flutter widgets
DI	Hilt	Koin	Manual / DependenciesScope
Learning curve (for you)	High (new)	Very high (new + KMP)	Low (you know it)
Job market (mobile dev)	Very strong	Growing fast	Strong
Best for learning Android	Yes	Not yet	No

15. Can This Project Be Configured for iOS? (KMP Migration)

Short answer: not with a configuration change. It would be a significant rewrite. Here is exactly what would need to change and how much work it is.

What would need to change

What changes	Effort	Why
Replace Hilt with Koin	High	Every @HiltViewModel, @Inject, @Module, @Binds, @Named -- all change. Every file in di/, controller/ needs updates.
Restructure Gradle to KMP	High	New plugins, new source sets (commonMain/androidMain/iosMain), new dependencies.
Set up Xcode project	Medium	iOS entry point, bundle config, signing certificates required.
Replace Jetpack Nav with KMP nav	Medium	Navigation Compose is Android-only. Need Voyager, Decompose, or Navigation 3.
Move shared code to commonMain	Medium	repositories, models, ViewModels move to shared/. Android-specific code stays in androidMain/.
Verify all libs have KMP versions	Low-Medium	Most libs (Coroutines, Ktor, Room/SQLDelight) have KMP support. Verify each one.
App.kt and MainActivity.kt	Low	Stay in androidMain/ unchanged.
models/ and data classes	None	Pure Kotlin, already KMP-compatible.
Coroutines and Flow	None	Already KMP-compatible.

[!] Recommendation: do not migrate this project to KMP. If you want KMP + iOS in the future, start a fresh KMP project with Koin from day one. Learn Android with this project first, then start fresh for KMP.

16. Full Summary and Learning Path Recommendation

Key rules to remember from this volume

Rule 1: ViewModels NEVER go in AppModule

@HiltViewModel handles ViewModel registration automatically.

AppModule is ONLY for repositories, API clients, databases.

Rule 2: ViewModel lifetime = NavBackStackEntry lifetime

Rotate device -> ViewModel SURVIVES

Navigate deeper -> ViewModel SURVIVES (still in back stack)

Press Back (pop) -> ViewModel DESTROYED (onCleared called)

Rule 3: Navigation approach by trigger type

User taps button -> Callback approach (Approach 1)

After async logic -> ViewModel Channel events (Approach 3)

Never pass NavController into a screen composable (Approach 2)

Rule 4: Channel vs StateFlow for navigation

StateFlow replays the last value to new collectors -> navigation fires twice on rotation

Channel delivers each event once and forgets it -> correct for one-shot navigation

Rule 5: This project is Android-only

Hilt is the main blocker for iOS/KMP

To support iOS: replace Hilt with Koin and restructure for KMP

Recommendation: learn Android thoroughly first, then consider KMP later

Suggested learning path

Phase 1 (now): Master Android fundamentals with this project

- Kotlin syntax and idioms
- Jetpack Compose UI
- ViewModel + StateFlow
- Hilt DI
- Navigation Compose
- Clean architecture (data/controller/widgets layers)

Phase 2 (after Phase 1): Go deeper on Android

- Room database (replace in-memory cart with persistent storage)
- Retrofit (replace fake data with a real REST API)
- DataStore (user preferences, auth tokens)
- SavedStateHandle (survive process death)
- Unit testing with MockK and Turbine
- UI testing with Compose Test

Phase 3 (future, optional): Kotlin Multiplatform

- Start a fresh KMP project (do not migrate this one)
- Replace Hilt with Koin from the start
- Use Ktor instead of Retrofit for KMP-compatible networking

- Use SQLDelight instead of Room for KMP-compatible database
- Add an iosApp Xcode target
- Share business logic, keep UI platform-specific or use Compose Multiplatform

The three volumes at a glance

Vol. 1 -- Project Setup to Full Feature Implementation

Build system (libs.versions.toml, build.gradle.kts)
Clean architecture layers (data/models/controller/widgets)
Hilt DI from App.kt to hiltViewModel()
Sealed classes and StateFlow for state management
Navigation Compose (NavHost, routes, path parameters)
All four features: Products, ProductDetail, Cart, Order

Vol. 2 -- DI Deep-Dive and State Persistence

ui/theme/ folder explained
How Hilt picks up AppModule at compile time (KSP chain)
Multiple implementations with @Named qualifier
FakeProductRepositoryImpl and its three uses
Five levels of state survival (remember -> Room)
Flutter vs Android state management comparison table

Vol. 3 -- Architecture Decisions and Cross-Platform (this volume)

ViewModels vs AppModule -- what goes where and why
ViewModel lifecycle tied to back stack
Three navigation patterns with code examples
Channel vs StateFlow for one-shot events
iOS limitations: Hilt is Android-only
Cross-platform options: KMP, Flutter, React Native
KMP migration cost analysis